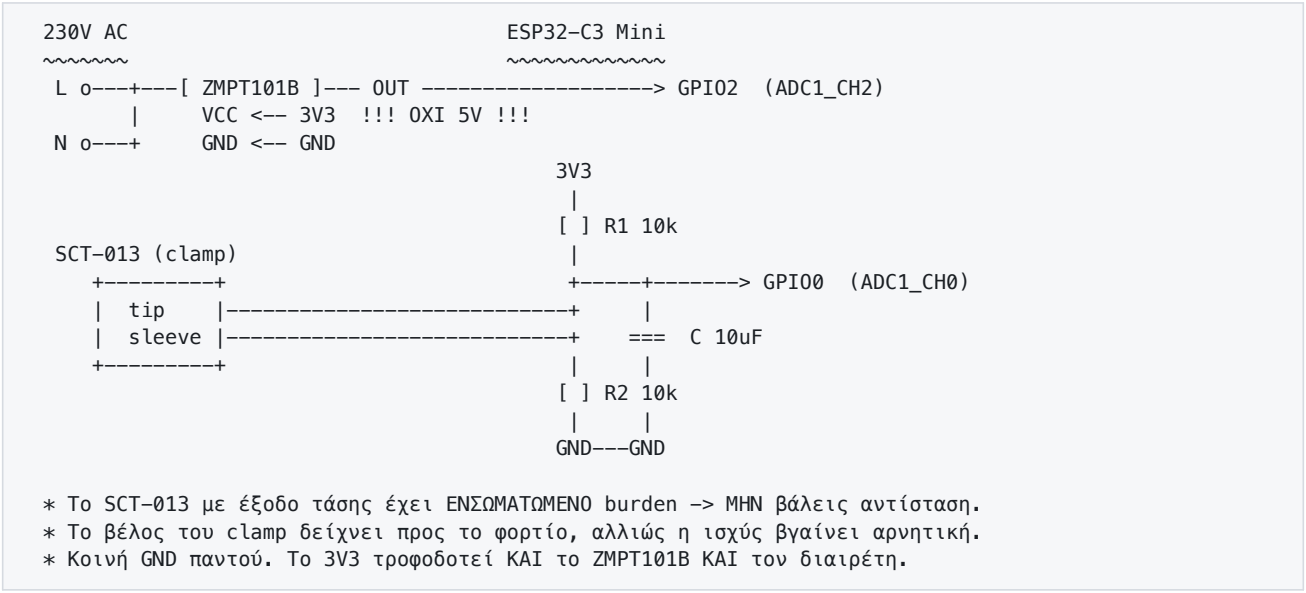
Smart Meter με ESP32-C3 Mini

Μέτρηση τάσης με ZMPT101B (GPIO2) και ρεύματος με SCT-013 (GPIO0). Υπολογίζει Vrms, Irms, πραγματική και φαινόμενη ισχύ, συντελεστή ισχύος και kWh, και τα δημοσιεύει σε MQTT με Home Assistant auto-discovery. **Οι τιμές βαθμονόμησης σε αυτό το έγγραφο είναι μετρημένες πάνω στην πραγματική διάταξη, όχι θεωρητικές.**

1. Συνδεσμολογία

Εξάρτημα	Ακροδέκτης	ESP32-C3	Σημείωση
ZMPT101B	VCC	**3V3**	**Κρίσιμο.** Με 5 V το σημείο ηρεμίας πάει στα 2.5 V, η κορυφή χτυπάει στο ταβάνι του ADC (2.85 V) και το ημίτονο κόβεται· επιπλέον καταπονείται το GPIO2.
ZMPT101B	OUT	GPIO2 (ADC1_CH2)	Το ποτενσιόμετρο ρυθμίζει το πλάτος — δες §3
ZMPT101B	GND	GND	Κοινή γείωση
SCT-013	σήμα (tip)	GPIO0 (ADC1_CH0)	Μέσο διαιρέτη 10k/10k → bias 1.65 V
SCT-013	sleeve	GND	Το βέλος του clamp προς το φορτίο
R1 / R2	10 kΩ	3V3 → GPIO0 → GND	Διαιρέτης πόλωσης· το 3V3 το μοιράζεται με το ZMPT101B
C	10 μF	GPIO0 → GND	Χαμηλή σύνθετη αντίσταση για το AC σήμα



2. Ρυθμίσεις Arduino IDE

- **Board:** ESP32C3 Dev Module (πακέτο esp32 by Espressif· δοκιμασμένο σε 3.3.10)
- **USB CDC On Boot: Enabled** — χωρίς αυτό ο Serial βγαίνει στα pins και ο monitor μένει κενός
- **Serial Monitor:** 115200 baud
- Η ίδια έξοδος βγαίνει και στο **UART0** (GPIO21=TX, GPIO20=RX) για μετατροπέα USB-TTL
- **Βιβλιοθήκη:** μόνο `PubSubClient` (Nick O'Leary). Οι υπολογισμοί δεν χρησιμοποιούν EmonLib.

3. Βαθμονόμηση — μετρημένες τιμές

Οι τέσσερις σταθερές στην αρχή του αρχείου. Οι τιμές που ακολουθούν προέκυψαν από σύγκριση με πολύμετρο και αμπεροτσιμπίδα πάνω στην πραγματική διάταξη.

Σταθερά	Τιμή	Πώς βρέθηκε
VCAL	**341.0**	Volt δικτύου ανά 1 V στο pin. Σύγκριση με πολύμετρο: ένδειξη 231.7 V έναντι 233 V → απόκλιση 0.55%. Αναπροσαρμογή: VCAL_νέο = VCAL_παλιό × (πολύμετρο ÷ ένδειξη).
ICAL	**25.75**	Ampere ανά 1 V στο pin. Θεωρητικά ένα SCT-013-030 θα ήθελε 30.0· το συγκεκριμένο clamp βγήκε **~25.8**. Τέσσερις συγκρίσεις σε 4.6 και 8.8 A έδωσαν 25.99 / 25.77 / 25.86 / 25.41 — μέση τιμή 25.76, διασπορά 1%.
PHASE_SHIFT	**12**	Διόρθωση φάσης σε **ολόκληρα δείγματα**. Ο μετασχηματιστής του ZMPT101B εισάγει **27.9°** μετατόπιση: χωρίς διόρθωση το PF βγαίνει 0.88 σε καθαρά ωμικό φορτίο και η ισχύς είναι **12% λάθος**. Βρίσκεται αυτόματα από τη σάρωση `[sweep]`.
PHASECAL	1.0	Λεπτή ρύθμιση μέσα σε ένα δείγμα (±2.5°). Δεν χρειάστηκε.

Σειρά εργασιών. Πρώτα το σχήμα, μετά η κλίμακα, τέλος η φάση:

- Ρύθμισε το ποτενσιόμετρο ώστε το `CF` να είναι **1.35–1.45** και το `max` να μένει **κάτω από 2800 mV**. Όσο το max είναι καρφωμένο στα 2853 το ημίτονο κόβεται, όποιο κι αν είναι το CF.
- Σύγκρινε την τάση με πολύμετρο και διόρθωσε το `VCAL`.
- Σύγκρινε το ρεύμα με αμπεροτσιμπίδα σε **δύο διαφορετικά φορτία** και διόρθωσε το `ICAL`. Δύο σημεία αποδεικνύουν γραμμικότητα, ένα όχι.
- Με **ωμικό** φορτίο άναψε τη σάρωση φάσης και πέρασε το προτεινόμενο `PHASE_SHIFT`.

 Το ZMPT101B συνδέεται στα 230 V AC. Κάθε επέμβαση με το κύκλωμα αποσυνδεδεμένο· για τη ρύθμιση του trimmer υπό τάση, μονωμένο κατσαβίδι και κλειστό κουτί.

4. Διαγνωστικά στη σειριακή

Με `PRINT_DEBUG 1` τυπώνονται τρία εργαλεία που φτιάχτηκαν ακριβώς για να στηθεί αυτή η διάταξη:

Γραμμή	Τι σου λέει
`[dbg]`	Τα δύο offsets (πρέπει και τα δύο ~1650 mV), ο αριθμός δειγμάτων (~2800 με σήμα, ~70 χωρίς), τα άκρα του καναλιού τάσης και το **crest factor**. Προειδοποιεί όταν η κορυφή ακουμπάει το ταβάνι του ADC.
`[I-only]`	Μέτρηση ρεύματος σε σταθερό παράθυρο 300 ms, ανεξάρτητη από την τάση. Τρέχει μόνη της όταν δεν υπάρχει σήμα τάσης, ώστε να ελέγχεις το clamp χωριστά. Δείχνει και το `ptp`: πραγματικό ημίτονο έχει ptp ≈ 2.83 × rms, ενώ ο θόρυβος 10–20 φορές το rms.
`[sweep]`	Καταγράφει 2048 δείγματα και δοκιμάζει 49 μετατοπίσεις φάσης. Με ωμικό φορτίο, η μετατόπιση με το μέγιστο PF είναι η σωστή. Τρέχει αυτόματα πάνω από 1 A.

5. MQTT

Συμπλήρωσε SSID, κωδικό και IP του broker στην αρχή του αρχείου. Με `USE_MQTT 0` ο μετρητής δουλεύει αυτόνομος. Το `<id>` είναι τα τρία τελευταία byte της MAC και τυπώνεται στην εκκίνηση.

Topic	Περιεχόμενο
`smartmeter/<id>/state`	JSON με όλες τις μετρήσεις, κάθε 5 s
`smartmeter/<id>/availability`	`online` / `offline` (retained, με Last Will)
`homeassistant/sensor/smartmeter_<id>/…/config`	Έξι μηνύματα auto-discovery, retained. Απενεργοποιούνται με `HA_DISCOVERY 0`

```
smartmeter/<id>/availability  online
smartmeter/<id>/state  {"v":232.61,"i":9.012,"p":2087.3,"s":2096.3,"pf":1.00,"e":0.1153,"rssi":-69}
```

Το Home Assistant φτιάχνει μόνο του μία συσκευή «Smart Meter» με έξι οντότητες. Η ενέργεια δηλώνεται `total_increasing`, οπότε μπαίνει στον Energy Dashboard.

6. Παράδειγμα εξόδου

```
================================================
 Smart Meter - ESP32-C3 | ZMPT101B(GPIO2) SCT-013(GPIO0)
================================================
 VCAL=341.00  ICAL=25.75  PHASECAL=1.00  PHASE_SHIFT=12
 Device ID : 8459a0
 MQTT topic: smartmeter/8459a0/state
 Εξοδος    : USB CDC + UART0 (GPIO21=TX, GPIO20=RX) @ 115200 baud
 [wifi] συνδεθηκε: IP=192.168.1.43  gateway=192.168.1.1  RSSI=-69 dBm
 [mqtt] συνδέθηκε στον 192.168.1.17
 [mqtt] στάλθηκε το Home Assistant discovery

V= 232.61 V | I= 9.012 A | P=  2087.3 W | S=  2096.3 VA | PF= 1.00 | E=    0.115 kWh
```

7. Κώδικας — smart_meter.ino

Τα στοιχεία WiFi και MQTT εμφανίζονται ως placeholder.

```
/* ============================================================================
   SMART METER  --  ESP32-C3 Mini
   ----------------------------------------------------------------------------
   Αισθητήρας τάσης  : ZMPT101B (module με τελεστικό + ποτενσιόμετρο) -> GPIO2
   Αισθητήρας ρεύματος: SCT-013 30A/1V (έξοδος τάσης, ΜΕ εσωτερικό burden)
                        -> GPIO0, με διαιρέτη τάσης 10k/10k (bias 1.65V)
                        και πυκνωτή 10uF από το μέσο του διαιρέτη προς GND.

   Μετράει: Vrms, Irms, Πραγματική Ισχύ (W), Φαινόμενη (VA), cos(φ), kWh
   και τα δημοσιεύει σε MQTT (με προαιρετικό Home Assistant auto-discovery).

   Απαιτεί τη βιβλιοθήκη: PubSubClient (Nick O'Leary) - Library Manager.

   !!! ΠΡΟΣΟΧΗ: το ZMPT101B συνδέεται στα 230V AC. Δούλεψε με το κύκλωμα
   ΑΠΟΣΥΝΔΕΔΕΜΕΝΟ από το δίκτυο. Μόνωση, κουτί, ασφάλεια.
   ============================================================================ */

// ------------------------------------------------------------ WiFi / MQTT
#define USE_MQTT      1      // 0 = μόνο σειριακή, χωρίς δίκτυο
#define HA_DISCOVERY  1      // 1 = αυτόματη αναγνώριση από Home Assistant

#if USE_MQTT
  #include <WiFi.h>
  #include <PubSubClient.h>

  static const char *WIFI_SSID = "TO_DIKTYO_MOU";
  static const char *WIFI_PASS = "TO_PASSWORD_MOU";

  static const char *MQTT_HOST = "192.168.1.17";   // IP του broker
  static const uint16_t MQTT_PORT = 1883;
  static const char *MQTT_USER = "O_XRHSTHS_MOU";        // "" αν δεν χρειάζεται
  static const char *MQTT_PASS = "TO_PASSWORD_TOU_BROKER";

  static const char *MQTT_BASE = "smartmeter";     // ρίζα των topics
  static const uint32_t PUBLISH_EVERY_MS = 5000;   // κάθε πότε δημοσιεύουμε
#endif

// ------------------------------------------------------------------ ΑΚΡΟΔΕΚΤΕΣ
static const int PIN_VOLT = 2;   // ZMPT101B  OUT  (ADC1_CH2)
static const int PIN_CURR = 0;   // SCT-013   sig  (ADC1_CH0)

// ------------------------------------------------------------ ΒΑΘΜΟΝΟΜΗΣΗ
// VCAL: πόσα Volt AC του δικτύου αντιστοιχούν σε 1 V στο pin.
//       Ξεκίνα με 260.0 και διόρθωσε: VCAL_new = VCAL_old * (Υπολύμετρου / Νσειριακής)
float VCAL = 341.0f;

// ICAL: πόσα Ampere αντιστοιχούν σε 1 V στο pin.
//       SCT-013-030 (30A/1V) απευθείας στο pin  -> 30.0
//       SCT-013-010 (10A/1V)                    -> 10.0
//       Αν έβαλες επιπλέον διαιρέτη ΠΑΝΩ στο σήμα (π.χ. /2) -> διπλασίασε το.
float ICAL = 25.75f;

// PHASECAL: διόρθωση διαφοράς φάσης V/I (1.0 = καμία διόρθωση).
//           Ρύθμισέ το με καθαρά ωμικό φορτίο ώστε cos(φ) ~ 1.00 (τυπικά 0.8 - 2.2)
float PHASECAL = 1.0f;

// PHASE_SHIFT: χονδρική διόρθωση φάσης σε ΟΛΟΚΛΗΡΑ δείγματα (το PHASECAL
// καλύπτει μόνο ±1 δείγμα = ±2.5 μοίρες, πολύ λίγο για τη μετατόπιση που
// εισάγει ο μετασχηματιστής του ZMPT101B). Θετικό = καθυστερεί την τάση.
```

```
// Βρες την τιμή με τη σάρωση [sweep] που τρέχει αυτόματα υπό φορτίο.
int PHASE_SHIFT = 12;

// ---------------------------------------------------------------- ΔΕΙΓΜΑΤΟΛΗΨΙΑ
static const int      HALF_CYCLES  = 40;     // 39 ημιπερίοδοι = 19.5 περίοδοι @50Hz (~390 ms)
static const uint32_t TIMEOUT_MS   = 1500;   // ασφάλεια αν δεν υπάρχει σήμα
static const double   FILT_TAU     = 1024.0; // σταθερά φίλτρου DC offset
static const float    NOISE_V      = 5.0f;   // κάτω από αυτό -> 0 (Volt)
static const float    NOISE_I      = 0.50f;  // κάτω από αυτό -> 0 (μετρημένος θόρυβος ~0.3 A)

// Αν το clamp είναι ανάποδα και δεν βολεύει να το γυρίσεις (π.χ. μόνιμη
// εγκατάσταση), βάλε 1 και αντιστρέφεται το πρόσημο της ισχύος από λογισμικό.
#define INVERT_CURRENT 0

#define PRINT_DEBUG 1                        // 1 = τύπωσε offsets/δείγματα

// ---------------------------------------------------------------- ΜΕΤΑΒΛΗΤΕΣ
static double offsetV = 1650.0;   // DC offset σε mV (ξεκινά στο μισό των 3.3V)
static double offsetI = 1650.0;

static double energy_Wh = 0.0;    // συσσωρευμένη ενέργεια
static uint32_t lastEnergy_ms = 0;

struct Reading {
  float  vrms;          // V
  float  irms;          // A
  float  realPower;     // W
  float  apparentPower; // VA
  float  powerFactor;   // 0..1
  int    samples;
  int    vmin, vmax;    // ακραίες τιμές του καναλιού τάσης, σε mV (έλεγχος clipping)
  bool   ok;
};

// ---------------------------------------------------------------- ΕΞΟΔΟΣ
// Τα μηνύματα βγαίνουν ταυτόχρονα σε δύο σημεία:
//   * USB CDC       -> ο Serial Monitor μέσω του καλωδίου προγραμματισμού
//   * hardware UART0 -> GPIO21 = TX, GPIO20 = RX, για μετατροπέα USB-TTL
// Έτσι βλέπεις την έξοδο ακόμη κι αν το USB CDC δεν είναι διαθέσιμο (π.χ.
// λάθος ρύθμιση "USB CDC On Boot", ή όταν το ESP32 τροφοδοτείται χωριστά).
#define LOG_TO_UART 1
static const uint32_t LOG_BAUD = 115200;

// Αν το USB CDC είναι απενεργοποιημένο, ο Serial ΕΙΝΑΙ ήδη ο UART0 -> μη
// γράψεις δύο φορές στο ίδιο πράγμα.
#if LOG_TO_UART && ARDUINO_USB_CDC_ON_BOOT
  #define LOG_DUAL 1
#else
  #define LOG_DUAL 0
#endif

static void logBegin() {
  Serial.begin(LOG_BAUD);
#if LOG_DUAL
  Serial0.begin(LOG_BAUD);          // GPIO21 TX / GPIO20 RX
#endif
  // Με USB CDC η θύρα ξαναδηλώνεται στο USB μετά το reset. Χωρίς αυτή την
  // αναμονή, τα πρώτα μηνύματα χάνονται πριν προλάβει να συνδεθεί ο monitor.
  while (!Serial && millis() < 3000) delay(10);
  delay(500);
}

static void logf(const char *fmt, ...) {
  char buf[224];
  va_list ap;
  va_start(ap, fmt);
  vsnprintf(buf, sizeof(buf), fmt, ap);
  va_end(ap);

  Serial.print(buf);
#if LOG_DUAL
  Serial0.print(buf);
#endif
}

// ---------------------------------------------------------------- ΜΕΤΡΗΣΗ
static Reading measure() {
  Reading r = {0, 0, 0, 0, 0, 0, 5000, 0, false};

```

```
  const uint32_t tStart = millis();

  // 1) Warm-up: αφήνουμε το φίλτρο DC offset να "κλειδώσει"
  while (millis() - tStart < 60) {
    offsetV += (analogReadMilliVolts(PIN_VOLT) - offsetV) / FILT_TAU;
    offsetI += (analogReadMilliVolts(PIN_CURR) - offsetI) / FILT_TAU;
  }

  // 2) Κύρια δειγματοληψία, συγχρονισμένη σε μηδενισμούς της τάσης
  const int PBUF = 32;                 // > |PHASE_SHIFT| + 1
  static double pbufV[32], pbufI[32];
  int tick = 0;

  double sumV = 0, sumI = 0, sumP = 0;
  double lastFilteredV = 0;
  int    crossCount = 0;
  int    n = 0;
  bool   started = false;

  // Αρχικοποίηση προσήμου από το τρέχον δείγμα, ώστε ο πρώτος μηδενισμός
  // που θα ανιχνευτεί να είναι πραγματικός -> αθροίζουμε ακέραιο αριθμό
  // ημιπεριόδων και το RMS βγαίνει ακριβές.
  bool   lastSign = (analogReadMilliVolts(PIN_VOLT) - offsetV) > 0;

  while (crossCount < HALF_CYCLES) {
    if (millis() - tStart > TIMEOUT_MS) return r;   // ok = false

    int    rawV = analogReadMilliVolts(PIN_VOLT);
    double sV = rawV;
    double sI = analogReadMilliVolts(PIN_CURR);

    if (rawV < r.vmin) r.vmin = rawV;
    if (rawV > r.vmax) r.vmax = rawV;

    offsetV += (sV - offsetV) / FILT_TAU;
    offsetI += (sI - offsetI) / FILT_TAU;

    double fV = sV - offsetV;      // AC συνιστώσα τάσης  (mV στο pin)
    double fI = sI - offsetI;      // AC συνιστώσα ρεύματος (mV στο pin)

    bool sign = (fV > 0);
    if (sign != lastSign) {
      crossCount++;
      if (!started) {              // ξεκίνα να αθροίζεις από τον 1ο μηδενισμό
        started = true;
        sumV = sumI = sumP = 0;
        n = 0;
      }
    }
    lastSign = sign;

    // Γράψε τα δείγματα στον κυκλικό buffer και διάβασε τα μετατοπισμένα.
    int pi = tick % PBUF;
    pbufV[pi] = fV;
    pbufI[pi] = fI;
    int kv = (PHASE_SHIFT > 0) ?  PHASE_SHIFT : 0;   // καθυστέρηση τάσης
    int ki = (PHASE_SHIFT < 0) ? -PHASE_SHIFT : 0;   // καθυστέρηση ρεύματος
    double v0 = pbufV[(pi - kv     + PBUF) % PBUF];
    double v1 = pbufV[(pi - kv - 1 + PBUF) % PBUF];
    double phaseShiftedV = v1 + PHASECAL * (v0 - v1);   // λεπτή ρύθμιση
    double iShifted      = pbufI[(pi - ki + PBUF) % PBUF];

    if (started && tick >= PBUF) {
      sumV += fV * fV;
      sumI += fI * fI;
      sumP += phaseShiftedV * iShifted;
      n++;
    }
    tick++;
    lastFilteredV = fV;
  }

  if (n < 10) return r;

  // 3) Υπολογισμοί (mV -> V, μετά επί τους συντελεστές βαθμονόμησης)
  double vrms_pin = sqrt(sumV / n) / 1000.0;   // V στο pin
  double irms_pin = sqrt(sumI / n) / 1000.0;   // V στο pin

  r.vrms      = VCAL * vrms_pin;
```

```
  r.irms      = ICAL * irms_pin;
  r.realPower = VCAL * ICAL * (sumP / n) / 1.0e6;

  if (r.vrms < NOISE_V) { r.vrms = 0; r.realPower = 0; }
  if (r.irms < NOISE_I) { r.irms = 0; r.realPower = 0; }

#if INVERT_CURRENT
  r.realPower = -r.realPower;
#endif

  r.apparentPower = r.vrms * r.irms;
  r.powerFactor   = (r.apparentPower > 0.5f) ? (r.realPower / r.apparentPower) : 0.0f;
  if (r.powerFactor >  1.0f) r.powerFactor =  1.0f;
  if (r.powerFactor < -1.0f) r.powerFactor = -1.0f;

  r.samples = n;
  r.ok = true;
  return r;
}

// ---------------------------------------------------------- ΔΙΚΤΥΟ MQTT
#if USE_MQTT

static WiFiClient   wifiClient;
static PubSubClient mqtt(wifiClient);

static char devId[8]     = "";   // τα 3 τελευταία byte της MAC, π.χ. "a1b2c3"
static char t_state[64]  = "";   // smartmeter/a1b2c3/state
static char t_avail[64]  = "";   // smartmeter/a1b2c3/availability
static uint32_t lastNetTry  = 0;
static uint32_t lastPublish = 0;

// Στοιχεία που δημοσιεύονται. Το "key" είναι και το πεδίο του JSON.
struct SensorDef {
  const char *key, *name, *unit, *devClass, *stateClass;
};
static const SensorDef SENSORS[] = {
  {"v",  "Τάση",               "V",   "voltage",        "measurement"},
  {"i",  "Ρεύμα",              "A",   "current",        "measurement"},
  {"p",  "Ισχύς",              "W",   "power",          "measurement"},
  {"s",  "Φαινόμενη ισχύς",    "VA",  "apparent_power", "measurement"},
  {"pf", "Συντελεστής ισχύος", "",    "power_factor",   "measurement"},
  {"e",  "Ενέργεια",           "kWh", "energy",         "total_increasing"},
};

static void netSetup() {
  WiFi.mode(WIFI_STA);
  WiFi.setSleep(false);                 // σταθερότερη σύνδεση, λιγότερα stalls
  WiFi.begin(WIFI_SSID, WIFI_PASS);

  uint8_t mac[6];
  WiFi.macAddress(mac);
  snprintf(devId,   sizeof(devId),   "%02x%02x%02x", mac[3], mac[4], mac[5]);
  snprintf(t_state, sizeof(t_state), "%s/%s/state",        MQTT_BASE, devId);
  snprintf(t_avail, sizeof(t_avail), "%s/%s/availability", MQTT_BASE, devId);

  mqtt.setServer(MQTT_HOST, MQTT_PORT);
  mqtt.setBufferSize(1024);             // τα payloads του discovery δεν χωράνε στα 256

  logf(" Device ID : %s\n", devId);
  logf(" MQTT topic: %s\n", t_state);
}

#if HA_DISCOVERY
// Στέλνει ένα retained "config" topic ανά μέγεθος, ώστε το Home Assistant να
// φτιάξει μόνο του τις οντότητες κάτω από μία συσκευή.
static void publishDiscovery() {
  char topic[96], payload[640], unitField[48];

  for (const SensorDef &s : SENSORS) {
    snprintf(topic, sizeof(topic),
             "homeassistant/sensor/%s_%s/%s/config", MQTT_BASE, devId, s.key);

    unitField[0] = '\0';
    if (s.unit[0]) snprintf(unitField, sizeof(unitField), "\"unit_of_meas\":\"%s\",", s.unit);

    snprintf(payload, sizeof(payload),
      "{\"name\":\"%s\",\"uniq_id\":\"%s_%s_%s\",\"stat_t\":\"%s\",\"avty_t\":\"%s\","
      "%s\"dev_cla\":\"%s\",\"stat_cla\":\"%s\",\"val_tpl\":\"{{ value_json.%s }}\","
```

```
      "\"dev\":{\"ids\":[\"%s_%s\"],\"name\":\"Smart Meter\",\"mf\":\"DIY\","
      "\"mdl\":\"ESP32-C3 + ZMPT101B + SCT-013\"}}",
      s.name, MQTT_BASE, devId, s.key, t_state, t_avail,
      unitField, s.devClass, s.stateClass, s.key,
      MQTT_BASE, devId);

    mqtt.publish(topic, payload, true);   // retained
  }
  logf(" [mqtt] στάλθηκε το Home Assistant discovery\n");
}
#endif

// Μη μπλοκάρουσα διαχείριση σύνδεσης: καλείται συνέχεια από τη loop().
static void netLoop() {
  if (WiFi.status() != WL_CONNECTED) {
    if (millis() - lastNetTry > 10000) {
      lastNetTry = millis();
      logf(" [wifi] επανασύνδεση...\n");
      WiFi.disconnect();
      WiFi.begin(WIFI_SSID, WIFI_PASS);
    }
    return;
  }

  // Μόλις συνδεθεί το WiFi, τύπωσε μία φορά τη διεύθυνση που πήρε.
  static bool ipShown = false;
  if (!ipShown) {
    ipShown = true;
    logf(" [wifi] συνδεθηκε: IP=%s  gateway=%s  RSSI=%d dBm\n",
         WiFi.localIP().toString().c_str(),
         WiFi.gatewayIP().toString().c_str(),
         WiFi.RSSI());
  }

  if (!mqtt.connected()) {
    if (millis() - lastNetTry > 5000) {
      lastNetTry = millis();
      char clientId[32];
      snprintf(clientId, sizeof(clientId), "%s_%s", MQTT_BASE, devId);

      // Last Will: αν χαθεί η συσκευή, ο broker στέλνει μόνος του "offline".
      bool ok = mqtt.connect(clientId,
                             MQTT_USER[0] ? MQTT_USER : nullptr,
                             MQTT_PASS[0] ? MQTT_PASS : nullptr,
                             t_avail, 0, true, "offline");
      if (ok) {
        logf(" [mqtt] συνδέθηκε στον %s\n", MQTT_HOST);
        mqtt.publish(t_avail, "online", true);
#if HA_DISCOVERY
        publishDiscovery();
#endif
      } else {
        logf(" [mqtt] αποτυχία (rc=%d)\n", mqtt.state());
      }
    }
    return;
  }

  mqtt.loop();
}

static void publishState(const Reading &r) {
  if (!mqtt.connected()) return;

  char payload[192];
  snprintf(payload, sizeof(payload),
    "{\"v\":%.2f,\"i\":%.3f,\"p\":%.1f,\"s\":%.1f,\"pf\":%.2f,\"e\":%.4f,\"rssi\":%d}",
    r.vrms, r.irms, r.realPower, r.apparentPower, r.powerFactor,
    energy_Wh / 1000.0, WiFi.RSSI());

  mqtt.publish(t_state, payload);
}

#endif  // USE_MQTT

// Μετράει ΜΟΝΟ ρεύμα, σε σταθερό παράθυρο χρόνου — δεν εξαρτάται από τους
// μηδενισμούς της τάσης. Χρήσιμο για να ελέγξεις το clamp μόνο του, πριν
// συνδέσεις το ZMPT101B στο δίκτυο. Τυπώνει και το πλάτος σε mV, ώστε να
// ξεχωρίζεις πραγματικό σήμα από θόρυβο.
```

```
static void currentOnlyReport(uint32_t windowMs) {
  const uint32_t t0 = millis();
  double sumI = 0;
  int n = 0, mn = 5000, mx = 0;

  while (millis() - t0 < windowMs) {
    int raw = analogReadMilliVolts(PIN_CURR);
    if (raw < mn) mn = raw;
    if (raw > mx) mx = raw;
    offsetI += (raw - offsetI) / FILT_TAU;
    double fI = raw - offsetI;
    sumI += fI * fI;
    n++;
  }

  float irms = (n > 9) ? (float)(ICAL * sqrt(sumI / n) / 1000.0) : 0.0f;
  logf("[I-only] Irms=%.3f A | ptp=%dmV (%d..%d) | %d δείγματα σε %ums -> %.1f kSPS\n",
       irms, mx - mn, mn, mx, n, windowMs, n / (float)windowMs);
}

// ------------------------------------------------------------ ΣΑΡΩΣΗ ΦΑΣΗΣ
// Καταγράφει ~290 ms ωμής κυματομορφής και υπολογίζει το PF για κάθε πιθανή
// μετατόπιση. Με ΩΜΙΚΟ φορτίο, η μετατόπιση που δίνει το μέγιστο PF είναι η
// σωστή τιμή για το PHASE_SHIFT.
#define SWEEP_N      2048
#define SWEEP_MAX_K  24

static void phaseSweep() {
  static float bufV[SWEEP_N], bufI[SWEEP_N];

  uint32_t t0 = micros();
  for (int n = 0; n < SWEEP_N; n++) {
    double sV = analogReadMilliVolts(PIN_VOLT);
    double sI = analogReadMilliVolts(PIN_CURR);
    offsetV += (sV - offsetV) / FILT_TAU;
    offsetI += (sI - offsetI) / FILT_TAU;
    bufV[n] = sV - offsetV;
    bufI[n] = sI - offsetI;
  }
  float fs = SWEEP_N / ((micros() - t0) / 1.0e6f);     // πραγματικός ρυθμός

  const int a = SWEEP_MAX_K, b = SWEEP_N - SWEEP_MAX_K;
  double sv = 0, si = 0;
  for (int n = a; n < b; n++) { sv += (double)bufV[n]*bufV[n]; si += (double)bufI[n]*bufI[n]; }
  double vr = sqrt(sv / (b - a)), ir = sqrt(si / (b - a));

  if (vr < 50.0 || ir < 20.0) { logf("[sweep] πολύ μικρό σήμα - βάλε φορτίο\n"); return; }

  // ΠΡΟΣΟΧΗ: η σάρωση δουλεύει πάνω σε ωμά δείγματα, ΧΩΡΙΣ να εφαρμόζει το
  // PHASE_SHIFT. Άρα το bestK είναι η απόλυτη τιμή που πρέπει να μπει εκεί,
  // όχι κάτι που προστίθεται στην τρέχουσα.
  int bestK = 0; double bestPF = 0, pf0 = 0, pfCur = 0;
  for (int k = -SWEEP_MAX_K; k <= SWEEP_MAX_K; k++) {
    double sp = 0;
    for (int n = a; n < b; n++) sp += (double)bufV[n - k] * bufI[n];
    double pf = sp / ((b - a) * vr * ir);
    if (k == 0)           pf0   = pf;
    if (k == PHASE_SHIFT) pfCur = pf;
    if (fabs(pf) > fabs(bestPF)) { bestPF = pf; bestK = k; }
  }

  logf("[sweep] fs=%.0fHz | αδιορθωτο(0)=%.3f | τρεχον(%d)=%.3f | ΒΕΛΤΙΣΤΟ PHASE_SHIFT=%d -> PF=%.3f (%.1f μοιρες)\n",
       fs, pf0, PHASE_SHIFT, pfCur, bestK, bestPF,
       bestK * 360.0f * 50.0f / fs);
  if (bestPF < 0) logf("[sweep] ΠΡΟΣΟΧΗ: αρνητικο PF -> το clamp ειναι αναποδα\n");
}

// ------------------------------------------------------------------ SETUP
void setup() {
  logBegin();

  analogReadResolution(12);                          // 0..4095
  analogSetPinAttenuation(PIN_VOLT, ADC_11db);       // πλήρες εύρος ~0..3.1V
  analogSetPinAttenuation(PIN_CURR, ADC_11db);

  logf("\n");
  logf("================================================\n");
  logf(" Smart Meter - ESP32-C3 | ZMPT101B(GPIO2) SCT-013(GPIO0)\n");
  logf("================================================\n");
```

```
  logf(" VCAL=%.2f  ICAL=%.2f  PHASECAL=%.2f\n", VCAL, ICAL, PHASECAL);
#if LOG_DUAL
  logf(" Εξοδος    : USB CDC + UART0 (GPIO21=TX, GPIO20=RX) @ %u baud\n", LOG_BAUD);
#else
  logf(" Εξοδος    : UART0 (GPIO21=TX, GPIO20=RX) @ %u baud\n", LOG_BAUD);
#endif
  logf("\n");

#if USE_MQTT
  netSetup();
#endif

  // Μια "τυφλή" μέτρηση για να σταθεροποιηθούν τα φίλτρα
  measure();
  lastEnergy_ms = millis();
}

// ------------------------------------------------------------------------ LOOP
void loop() {
#if USE_MQTT
  netLoop();
#endif

  Reading r = measure();

  uint32_t now = millis();
  float dt_h = (now - lastEnergy_ms) / 3600000.0f;   // ms -> ώρες
  lastEnergy_ms = now;

  if (!r.ok) {
    logf("[!] Δεν ανιχνεύτηκε σήμα τάσης (timeout). Ελέγξτε ZMPT101B / GPIO2.\n");
    delay(1000);
    return;
  }

  energy_Wh += r.realPower * dt_h;

  // Χωρίς σήμα τάσης η μέτρηση ρεύματος δεν είναι αξιόπιστη (το παράθυρο
  // ορίζεται από τους μηδενισμούς της τάσης) -> κάνε ξεχωριστή μέτρηση.
  if (r.vrms <= 0.0f) currentOnlyReport(300);

#if PRINT_DEBUG
  // Με φορτίο πάνω από 1 A, τρέξε σάρωση φάσης κάθε 10 δευτερόλεπτα.
  static uint32_t lastSweep = 0;
  if (r.irms > 1.0f && millis() - lastSweep > 10000) {
    lastSweep = millis();
    phaseSweep();
  }
#endif

#if PRINT_DEBUG
  // Crest factor = κορυφή / ενεργός τιμή. Καθαρό ημίτονο -> 1.41.
  // Πολύ μικρότερο σημαίνει ότι το σήμα "πλακώνει" (saturation του τελεστικού).
  float vrmsPin_mV = (VCAL > 0) ? (r.vrms / VCAL * 1000.0f) : 0.0f;
  float crest = (vrmsPin_mV > 1.0f) ? ((r.vmax - r.vmin) / 2.0f) / vrmsPin_mV : 0.0f;

  logf("[dbg] offsetV=%.0fmV offsetI=%.0fmV samples=%d | V: %d..%d mV ptp=%d CF=%.2f %s\n",
       offsetV, offsetI, r.samples, r.vmin, r.vmax, r.vmax - r.vmin, crest,
       // Το ταβάνι του ADC είναι ~2850 mV: αν το ακουμπάμε, κόβεται η κορυφή
       // ΑΝΕΞΑΡΤΗΤΑ από το τι δείχνει το crest factor.
       (r.vmax > 2800 || r.vmin < 150) ? "<<< ΚΟΡΥΦΗ ΣΤΟ ΤΑΒΑΝΙ - χαμηλωσε κι αλλο"
         : (crest > 1.33f && crest < 1.50f) ? "OK"
         : (crest > 0.0f ? "<<< σχημα ακομα στραβο" : ""));
#endif

  logf("V=%7.2f V | I=%6.3f A | P=%8.1f W | S=%8.1f VA | PF=%5.2f | E=%9.3f kWh\n",
                r.vrms, r.irms, r.realPower, r.apparentPower,
                r.powerFactor, energy_Wh / 1000.0);

#if USE_MQTT
  if (millis() - lastPublish >= PUBLISH_EVERY_MS) {
    lastPublish = millis();
    publishState(r);
  }
#endif

  delay(200);
}
```